Notas de Aula - Org Arq



Ariel Alves da Silva



Academic year 2026

Índice

Índice 2
1. Primeira Parte 4
1.1. Arquitetura de Von-Neumann 4
CPU 4
Registers 4
Bus 4
1.2. Ciclo de instrução 5
Ciclo de busca 5
Ciclo de execução 5
1.3. Arquitetura RISC-V 5
Compilando .asm 5
Estrutura de código 5
Diretivas 6
Registradores 6
Comandos 6
Ecalls 7
Alocando na stack 7
Instruction Set Architecture 8
Formato das instruções (6 tipos) 8
Tipo R (register) 8
Tipo I (immediate) 8
Tipo S (Store) 9
Tipo B (branch) 9
Tipo U (Upper immediate) 9
Tipo J 9
1.4. Monociclo (RISC-V) 9
Metodologia de Clocking 10
Componentes 10
Arquitetura do banco de registradores 10
Arquitetura da ALU 10
Arquitetura de operações com memória 11

Execução de instruções 11
Execução das instruções tipo R 11
Execução de load/store 12
Execução de Branch 12
Arquitetura completa 13
Tabelas e Blocos Auxiliares 14
1.5. Notas avulsas 14
−4 em binário (20 bits) 14
Formato de números inteiros 15
2. Revisão de Lógica digital 16
2.1. Circuitos combinacionais e sequenciais 16
2.2. Identidades básicas 16
2.3. Mapas de Karnaugh 16
2.4. Flip Flops 17
Flip-Flop S-R 17
Flip-Flop J-K 17
2.5. Registradores 18
3. Segunda Parte 19
3.1. Multiciclo 19
3.2. Pipeline 19
Etapas 19
Comparação entre Monociclo, Multiciclo e Pipeline 20
Considerações 21
Registradores intermediários do pipeline 21

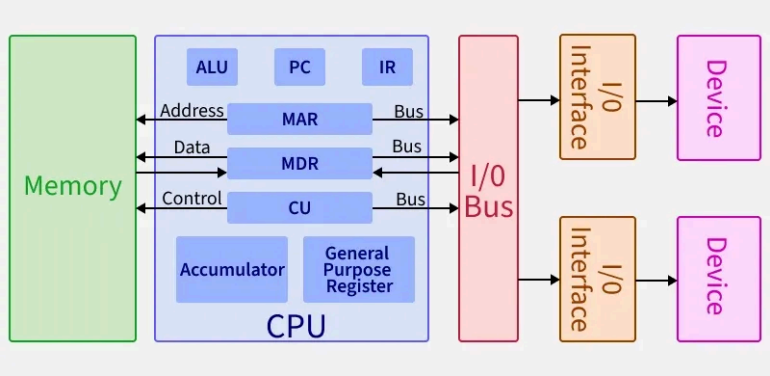
1. Primeira Parte

1.1. Arquitetura de Von-Neumann

Design de computador onde as instruções e dados são armazenados no mesmo espaço de memória. Os componentes são:

- CPU (CU + ALU)
- Registradores armazenados na CPU
- Dispositivos I/O operados pela CU
- Memória

CPU



- **CU (Control Unit)** - Gerencia como o processador funciona mandando sinais de controle (0/1) ao decodificar instruções.
- **ALU (Arithmetic and Logic Unit)** - Parte da CPU que lida com cálculos. Performa adição, subtração e operações lógicas, como comparações e deslocamento de bits em dados.

Registers

Tipo mais rápido de memória localizado na CPU.

- **PC (Program Counter)** - Endereço da próxima instrução a ser executada
- **IR (Instruction Register)** - Mantém a instrução atual sendo executada
- **MAR (Memory Address Register)** - Armazena o endereço da localização de memória sendo acessada
- **MDR (Memory Data Register)** - Armazena dados transferidos da/para a memória
- **Accumulator** - Armazena resultados intermediários de aritmética e operações lógicas
- **General Purpose Registers** - Uso para armazenamento temporário de dados

Bus

Sistema de comunicação que transfere dados, endereços, e sinais de controle entre a CPU, memória e dispositivos I/O. Na arquitetura de Von Neumann, um único bus é compartilhado para ambos dados e instruções.

1.2. Ciclo de instrução

Dois passos:
- Ciclo de busca (fetch)
- Ciclo de execução (execute)

Ciclo de busca

- PC tem endereço da próxima instrução
- Processador busca instrução na posição de memória armazenada no PC
  - MAR = PC
  - MBR = memória(MAR)
- PC = PC+4 (exceto em branch)
- Instrução armazenada no IR - IR = MBR

Ciclo de execução

- CU decodifica instrução e define sinais de controle
- Execução da instrução se resume a uma das seguintes possibilidades:
  - **Processador-memória**: Transferir dados do processador para a memória ou vice-versa
  - **Processador-I/O**: Transferir dados entre processador e dispositivo I/O
  - **Processamento de dados**: Executar operações aritméticas/lógicas sobre os dados
  - **Controle**: Especifica que a sequência de execução de instruções seja alterada
  - **Combinação dessas 4 possibilidades**

1.3. Arquitetura RISC-V

- Arquitetura de 32 bits
- 32 registradores de propósito geral
- 32 registradores para float

Compilando .asm

1. `sudo apt install nasm`
2. `sudo apt install binutils`
3. `nasm -f elf64 file.asm -o file.o` (monta arquivo)
4. `ld file.o -o file` (cria executável)

Estrutura de código

- `.data` - Início da seção de dados estáticos
- `.text` - Início da seção de código
- `.global label` - Entry point. Permite que outros arquivos chamem essa função durante o processo de link.

Diretivas

- `.align n` - Alinha a memória em 2^n bytes. Força o próximo dado/instrução a começar em um endereço de memória alinhado, utilizando padding.
  - `n = 0` para strings.
  - `n = 2` para instruções.
- `label: .asciz "str"` - Guarda string e encerra ela com `\0`
- `label: .byte b1,...,bn` - Reserva e inicializa cada byte sequencialmente.
- `label: .word w1,...,wn` - Reserva e inicializa cada word sequencialmente.
- `label: .double d1,...,dn` - Reserva e inicializa cada double sequencialmente.
- `label: .space n` - Reserva n bytes

Registradores

- `ra` - Return address
- `sp` - Stack pointer
- `t0-t6` - Temporary
- `a0-a6` - Function arguments
- `a7` - Syscall register
- `s0-s11` - Saved values

Comandos

Referência: Risc-V Cheat Sheet

Aritmética

- `add rd, rs1, rs2` - `rd = rs1 + rs2` (2 registradores)
- `addi rd, rs1, imm` - `rd = rs1 + imm` (1 registrador, 1 valor imediato/constante)
- `sub rd, rs1, rs2` - `rd = rs1 - rs2`
- `mul rd, rs1, rs2` - `rd = rs1 * rs2`
- `div rd, rs1, rs2` - `rd = rs1 / rs2`
- `rem rd, rs1, rs2` - `rd = rs1 % rs2`
- `remu rd, rs1, rs2` - `rd = rs1 % rs2` (unsigned remainder)
- `xor rd, rs1, rs2` - `rd = rs1 xor rs2`

Load/Store

- `la rd, label` - `rd = label_addr`
- `lw rd, imm(rs1)` - `rd = mem[rs1+imm]` (word = 4 bytes)
- `lb rd, imm(rs1)` - `rd = mem[rs1+imm]` (byte)
- `sw rs2, imm(rs1)` - `mem[rs1+imm] = rs2`
- `sb rs2, imm(rs1)` - `mem[rs1+imm] = rs2`

Jump/Function

- `j label` - `pc += label` (Obs: Usa half-words para pular. `jmp 4` -> `pc += 4 * 2 bytes`)
- `jal rd, label` - `rd = pc+4; pc += label` (salva endereço de retorno)
- `jr ra` - `pc = ra`

- `call symbol` - `ra = pc+4; pc = &symbol`
- `ret` - `pc = ra`

Branch (imm pode também ser um label)

- `beq rs1, rs2, imm` - `if(rs1 == rs2) pc += imm`
- `bne rs1, rs2, imm` - `if(rs1 != rs2) pc += imm`
- `blt rs1, rs2, imm` - `if(rs1 < rs2) pc += imm`
- `ble rs1, rs2, imm` - `if(rs1 <= rs2) pc += imm`
- `bgt rs1, rs2, imm` - `if(rs1 > rs2) pc += imm`
- `bge rs1, rs2, imm` - `if(rs1 >= rs2) pc += imm`
- `bnez rs1, imm` - `if(rs1 != 0) pc += imm`

Comparadores

- `slt rd, rs1, rs2` - `rd = (rs1 < rs2)`
- `slti rd, rs1, imm` - `rd = (rs1 < imm)`

Ecalls

- `a7` - Armazena número da ecall
- `ecall` - Chama a ecall

Chamadas ao sistema (ecall)

Name	Number	Description	Inputs	Ouputs
PrintInt	1	Prints an integer	a0 = integer to print	N/A
PrintFloat	2	Prints an floating point number	fa0 = float to print	N/A
PrintDouble	3	Prints a double precision floating point number	fa0 = double to print	N/A
PrintString	4	Prints a null-terminated string to the console	a0 = the address of the string	N/A
ReadInt	5	Read an int from input console	N/A	a0 = the int
ReadFloat	6	Read a float from input console	N/A	fa0 = the float
ReadDouble	7	Reads a double from input console	N/A	fa0 = the double
ReadString	8	Reads a string from the console	a0 = address of input buffer a1 = maximum number of characters to read	N/A
Sbrk	9	Allocate heap memory	a0 = amount of memory in bytes	a0 = address to the allocated block
Exit	10	Exits the program with code 0	N/A	N/A
PrintChar	11	Prints an ascii character	a0 = character to print (only lowest byte is considered)	N/A
ReadChar	12	Read a character from input console	N/A	a0 = the character
Exit2	93	Exits the program with a code	a0 = the number to exit with	N/A

Alocando na stack

Empilhando

- `addi sp, sp, -8` - Reserva 8 words
- `sw a0, 4(sp)` - Salva `a0` na stack (sp+4)
- `sw ra, 0(sp)` - Salva `ra` na stack (sp)

Desempilhando

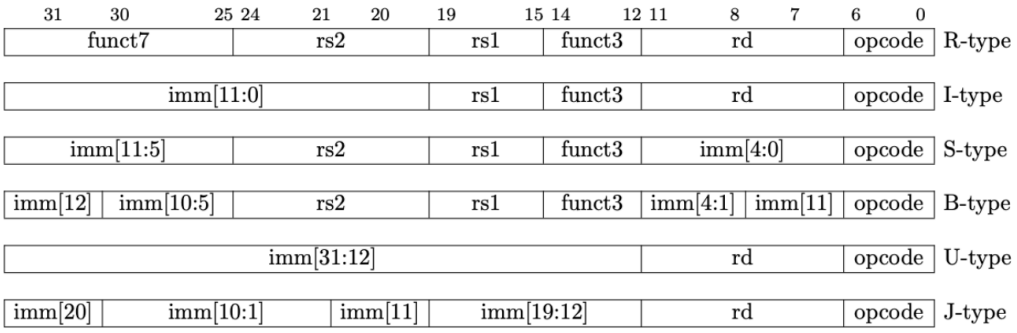
- `lw a0, 4(sp)`
- `lw ra, 0(sp)`
- `addi sp, sp, 8`

Instruction Set Architecture

Modelo abstrato que define a interface programável da CPU do computador, definindo como o software interage com o hardware. Um dispositivo que interpreta as instruções descritas por uma ISA é uma implementação da ISA.

- Quantidade e função dos registradores
- Formato das instruções (6 tipos)

Formato das instruções (6 tipos)



Tipo R (register)

Operações matemáticas e lógicas: `add`, `sub`, `and`, `or`, `sll`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `rs2` - Endereço do segundo registrador de origem
- `funct7` - Auxílio para definição da operação

Ex: `add rd1, rs1, rs2` -> `0000000 01000 01001 000 10010 0110011`

Tipo I (immediate)

Operações aritméticas com constantes ou instruções de leitura da memória: `addi`, `lw`, `andi`, `jalr`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem

- `imm` - Valor imediato

Ex: `lw s2, 0(sp)` -> `000000000000 00010 010 10010 0000011`

Tipo S (Store)

Escrever dados de um registrador para a memória: `sw`

- `opcode` - Código da operação
- `imm[4:0]` e `imm[11:5]` - Valor imediato (Separa os bits)
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `rs2` - Endereço do segundo registrador de origem

Ex: `sw s2, 0(sp)` -> `00000000 10010 00010 010 00000 0100011`

Tipo B (branch)

Desvios condicionais: `beq`, `bne`, etc

- `opcode` - Código da operação
- `imm[4:0]`, `imm[11]`, `imm[11:5]`, `imm[12]` - Valor imediato
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `rs2` - Endereço do segundo registrador de origem

Ex: `beq s1, s0, 4` -> `0 000000 01000 01001 000 0100 0 1100011`

Tipo U (Upper immediate)

Carregar números muito grandes diretamente nos bits superiores de um registrador: `lui`, `auipc`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `imm[31:12]` - Valor imediato

Ex: `lui s0, 0x01234` -> `[0000 0001 0010 0011 0100] 01000 0110111`

Tipo J

Pulos incondicionais de longo alcance (chamadas de função e desvios obrigatórios): `jal`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `imm[20]`, `imm[10:1]`, `imm[11]`, `imm[19:12]` - Valor imediato

Ex: `jal s0, -4` -> `1 1111111100 1 11111111 01000 1101111`

1.4. Monociclo (RISC-V)

A instrução é executada em um único ciclo de clock. Final de ciclo: tudo o que está nas conexões (linhas) é salvo nos registradores.

Componentes: PC, Banco registradores, ULA, UC, Memória, I/O.

Conjunto reduzido de instruções:

- Tipo R: `add`, `sub`, `and`, `or`

- Tipo I: `lw`
- Tipo S: `sw`
- Tipo B: `beq`

Metodologia de Clocking

Quaisquer valores armazenados em um elemento lógico sequencial são atualizados apenas em uma transição de clock: `State 1 -> Combinational Logic -> State 2`, em apenas 1 ciclo de clock.

Componentes

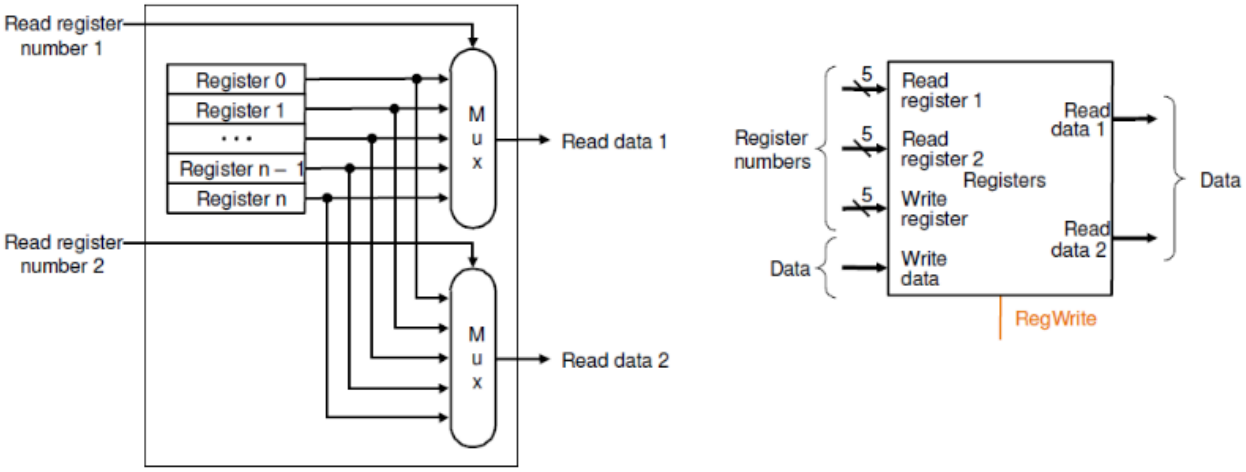
Arquitetura do banco de registradores

READ

1. 32 registradores
2. A saída de cada registrador vai para 2 MUX (saída duplicada)
3. 5 bits entram no Select do primeiro MUX -> Read Register 1 (5 bits)
4. 5 bits entram no Select do segundo MUX -> Read Register 2 (5 bits)
5. Saída do primeiro MUX -> Read Data 1 (32 bits)
6. Saída do segundo MUX -> Read Data 2 (32 bits)

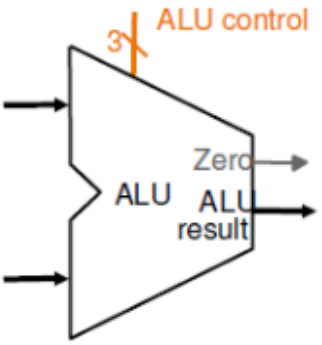
WRITE

1. 32 registradores
2. Entrada Write Register (5 bits)
3. Entrada Write Data (32 bits)



Arquitetura da ALU

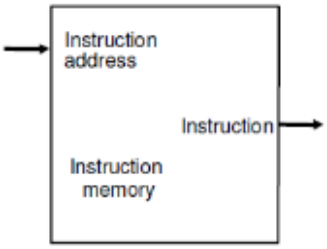
1. Recebe 2 entradas (32 bits)
2. Devolve saída (32 bits)



Arquitetura de operações com memória

Mem. Instrução:

1. Recebe endereço do PC (32 bits)
2. Devolve instrução (32 bits)

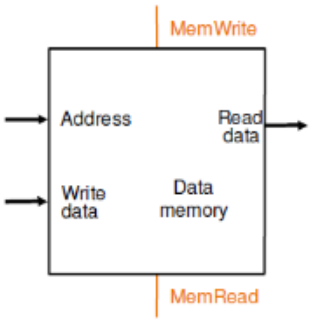


Mem. Dados:

1. Recebe endereço da ULA (32 bits) -> Devolve Read Data (32 bits)

ou

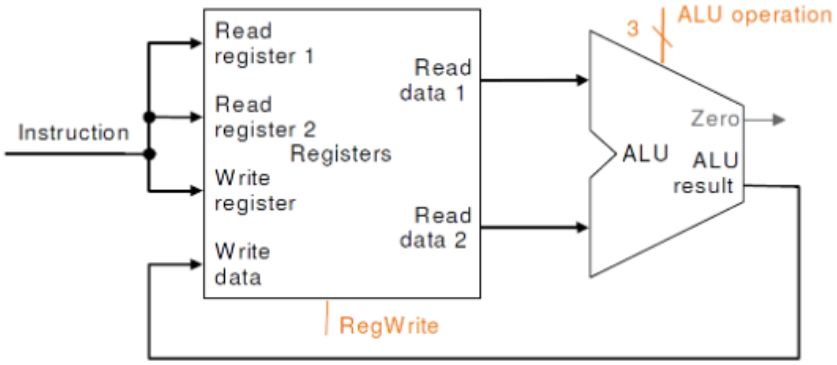
1. Recebe endereço (32 bits) -> Escreve Write Data (32 bits)



Execução de instruções

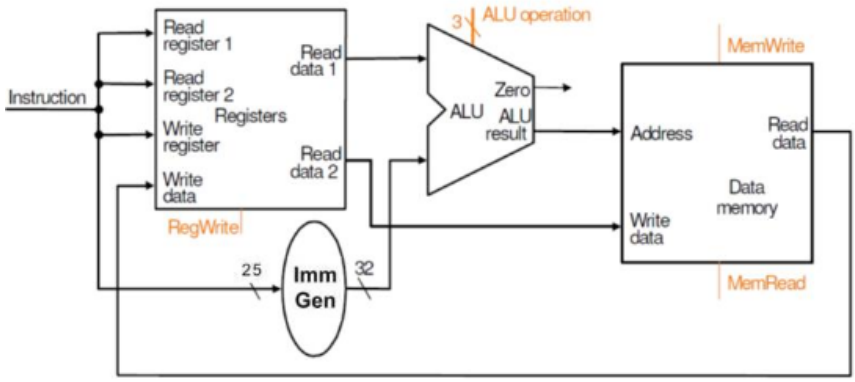
Execução das instruções tipo R

- Leitura de dois registradores
- Realização de operação aritmética/lógica
- Escrita do resultado no registrador



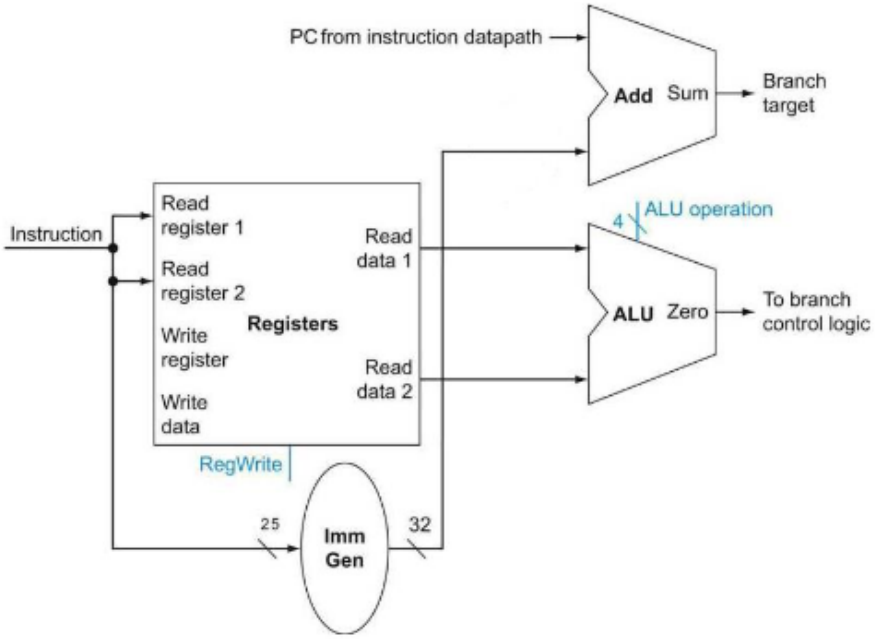
Execução de load/store

- **Load**: Leitura de registrador origem e valor imediato
  - Calcula endereço usando deslocamento de 12 bits (usa ALU)
  - Lê a memória e atualiza o registrador destino.
- **Store**: Leitura de dois registradores origem e valor imediato
  - Calcula endereço usando deslocamento de 12 bits (usa ALU)
  - Escreve o valor do registrador na memória.

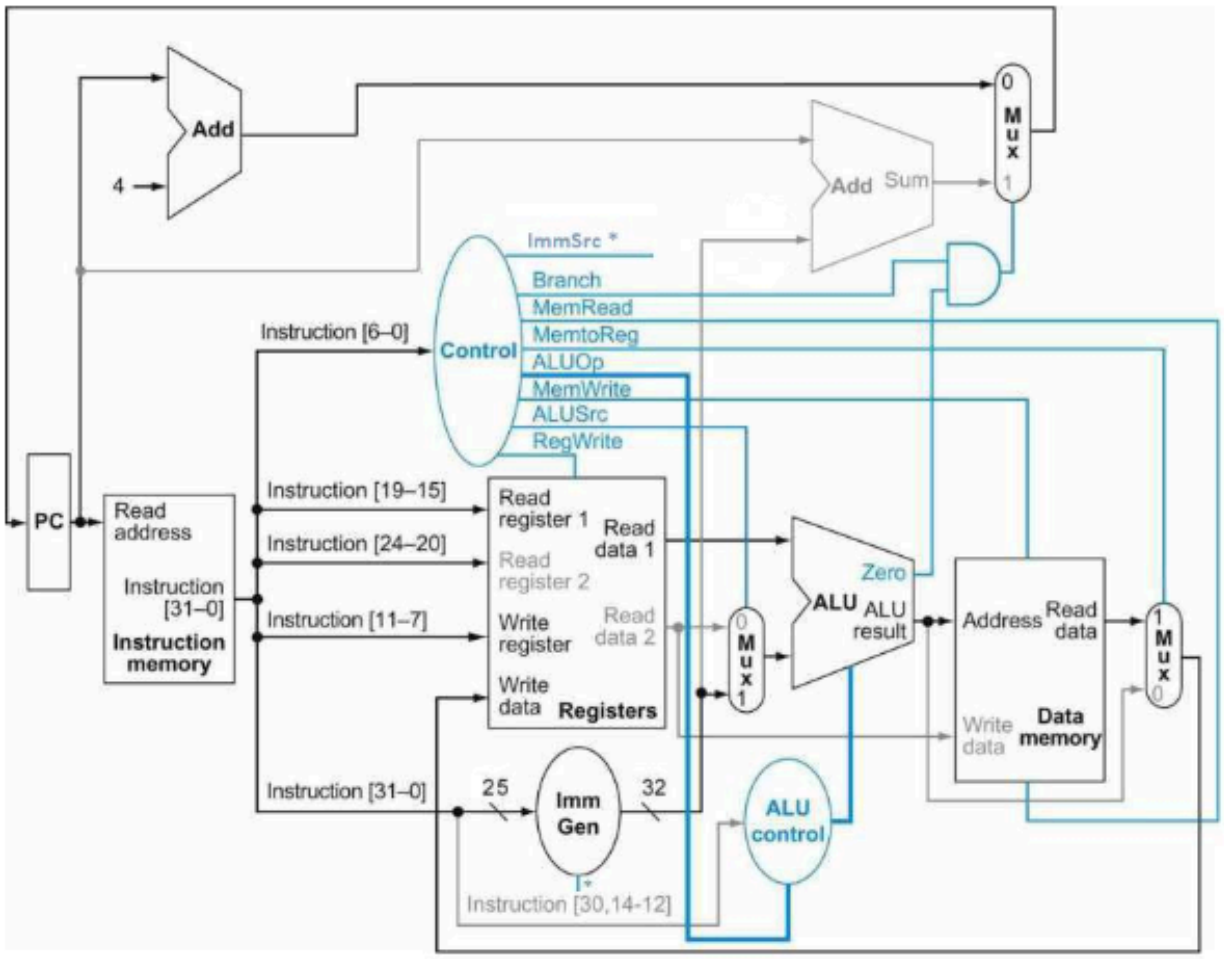


Execução de Branch

- Leitura de registradores
- Comparação dos dois através da ALU (se forem iguais, `zero = 1`)
- Endereço destino = PC + offset



Arquitetura completa

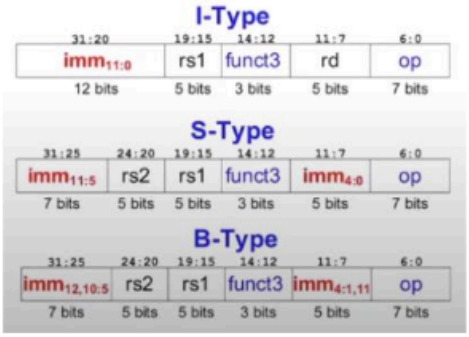


Tabelas e Blocos Auxiliares

ALU control lines	Function
000	AND
001	OR
010	add
110	subtract

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	010
sw	00	store word	XXXXXXX	XXX	add	010
beq	01	branch if equal	XXXXXXX	XXX	subtract	110
R-type	10	add	0000000	000	add	010
R-type	10	sub	0100000	000	subtract	110
R-type	10	and	0000000	111	AND	000
R-type	10	or	0000000	110	OR	001

$ImmSrc_{1:0}$	ImmExt	Instruction Type
00	{{20{instr[31]}}, instr[31:20]}	I-Type
01	{{20{instr[31]}}, instr[31:25], instr[11:7]}	S-Type
10	{{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0}	B-Type



Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0	ImmSrc
R-format	0	0	1	0	0	0	1	0	XX
lw	1	1	1	1	0	0	0	0	00
sw	1	X	0	0	1	0	0	0	01
beq	0	X	0	0	0	1	0	1	10

1.5. Notas avulsas

–4 em binário (20 bits)

Ao invés de usar bit de sinal, podemos usar complemento de dois: Inverte e soma 1.

```
0000000000000000100 -> 4
1111111111111111011 -> Complemento
+                   1
--------------------
1111111111111111100 -> -4
```

Formato de números inteiros

- 1 bit de sinal, 15 bits de magnitude.

2. Revisão de Lógica digital

2.1. Circuitos combinacionais e sequenciais

- Circuitos combinacionais - Função apenas de sinais de entrada.
- Circuitos sequenciais - Sinal de saída depende dos sinais anteriores de saída.

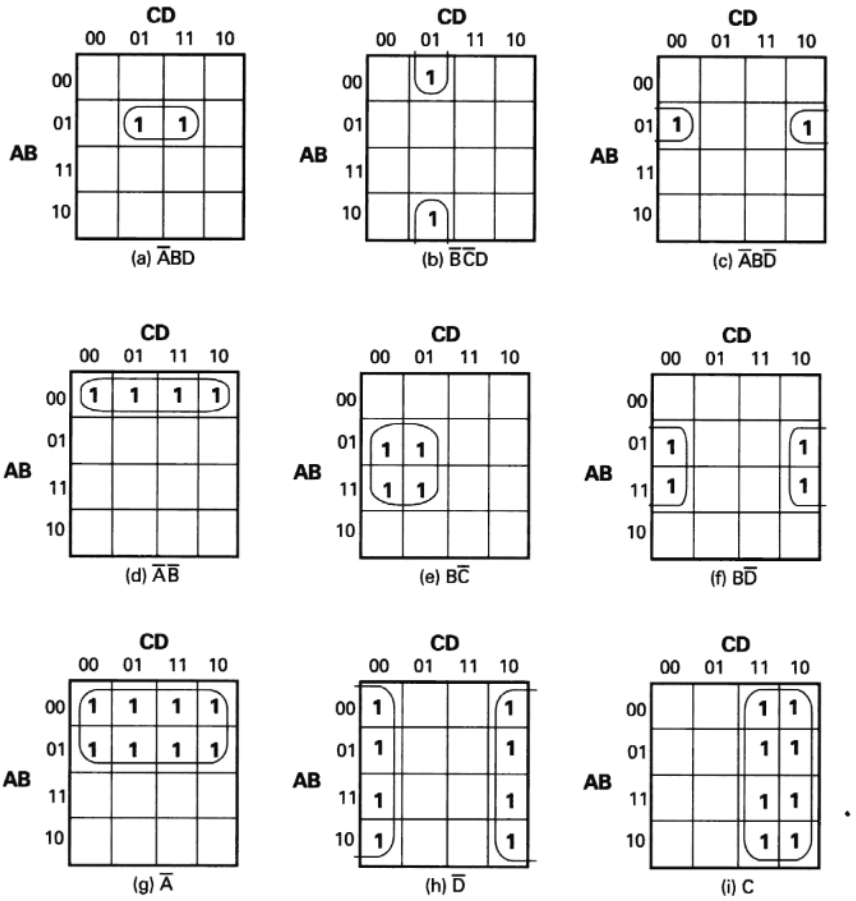
2.2. Identidades básicas

Basic Postulates		
$A \cdot B = B \cdot A$	$A + B = B + A$	Commutative Laws
$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$	Distributive Laws
$1 \cdot A = A$	$0 + A = A$	Identity Elements
$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$	Inverse Elements
Other Identities		
$0 \cdot A = 0$	$1 + A = 1$	
$A \cdot A = A$	$A + A = A$	
$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$	Associative Laws
$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$	DeMorgan's Theorem

2.3. Mapas de Karnaugh

Bits em ordem de Gray code (apenas 1 bit muda de estado a cada incremento/decremento).

Soma de produtos: Agrupar 1s em clusters de 1, 2, 4, 8, ...



== Principais circuitos

- Half Adder - 2 entradas, 1 saída, 1 Cout.
- Full Adder - 2 entradas, 1 Cin, 1 saída, 1 Cout.
- Demux - Entrada de n bits, saída de 2^n bits. Somente uma saída é ativada para cada combinação de entradas.
- Mux - Até 2^n entradas e n valores seletores. Direciona uma das entradas para a saída.

2.4. Flip Flops

São circuitos sequenciais.

Flip-Flop S-R

- Duas entradas: S (set) e R (reset)
- Duas saídas: Q e $|Q|$
- $S = R = 1$ deve ser evitada. Resolve-se com flip-flop de entrada única (Flip-Flop D).

Flip-Flop J-K

- $J = K = 0$ - Mantém
- $J = 1, K = 0$ - Saída = 1
- $J = 0, K = 1$ - Saída = 0
- $J = K = 1$ - Inverte

2.5. Registradores

Exemplo de uso de flip-flops.

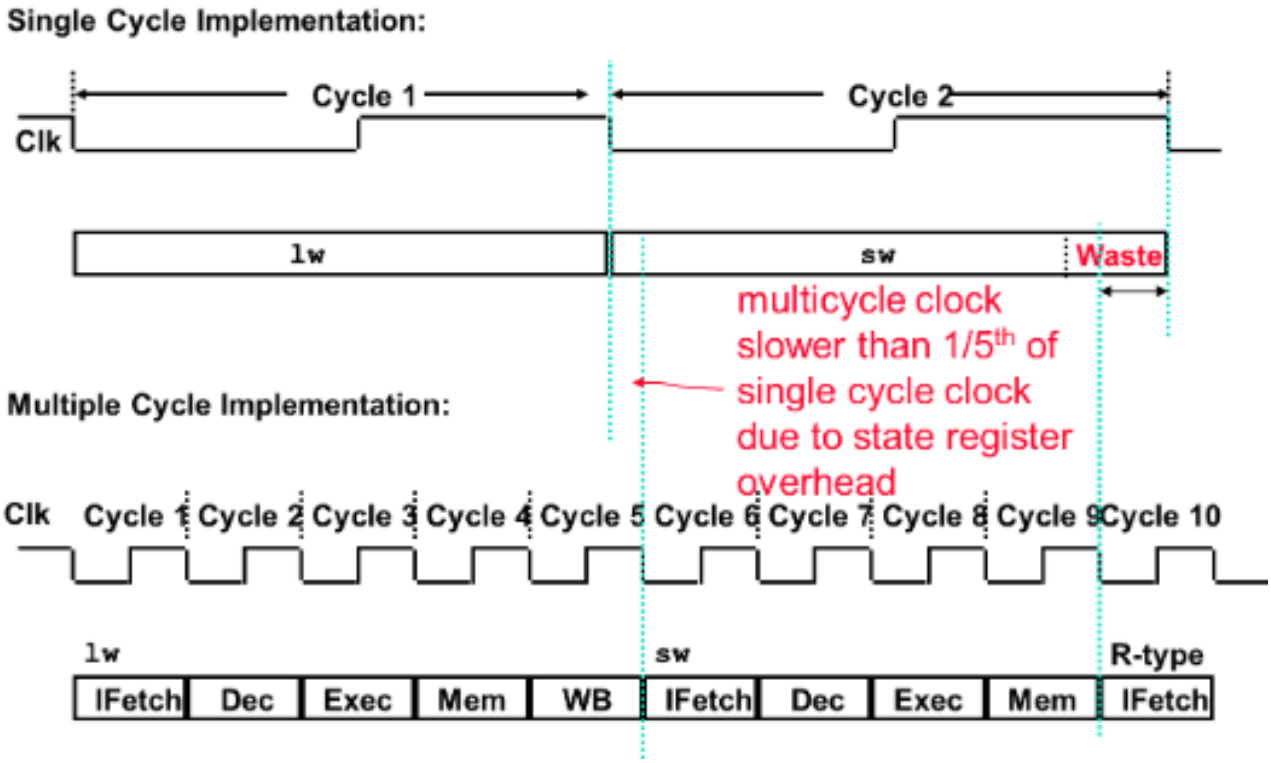
- Registradores paralelos - Carregamento e descarregamento feito simultaneamente.
- Registradores de deslocamento - Transfere informações serialmente. Uso em I/O e também dentro da ALU (deslocamento lógico e rotação).
- Contadores - Registrador cujo valor é incrementado de 1 módulo a capacidade d

3. Segunda Parte

3.1. Multiciclo

A desvantagem do monociclo é que o uso do ciclo de clock se torna ineficiente, pois deve ser do tamanho da instrução mais lenta. Assim, instruções rápidas ficam aguardando tempo demais pelo fim do ciclo de clock.

No multiciclo, os ciclos de clock são menores, e cada ciclo de clock é ume estágio da instrução: IFech, Dec, Exec, Mem, WB. Nem todas as instruções demoram o mesmo número de ciclos de clock para finalizar.

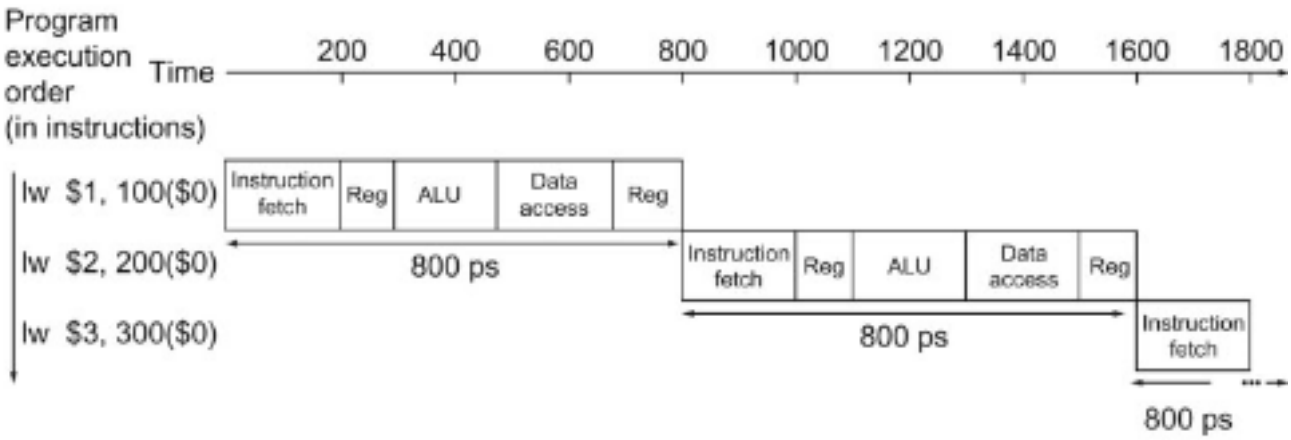


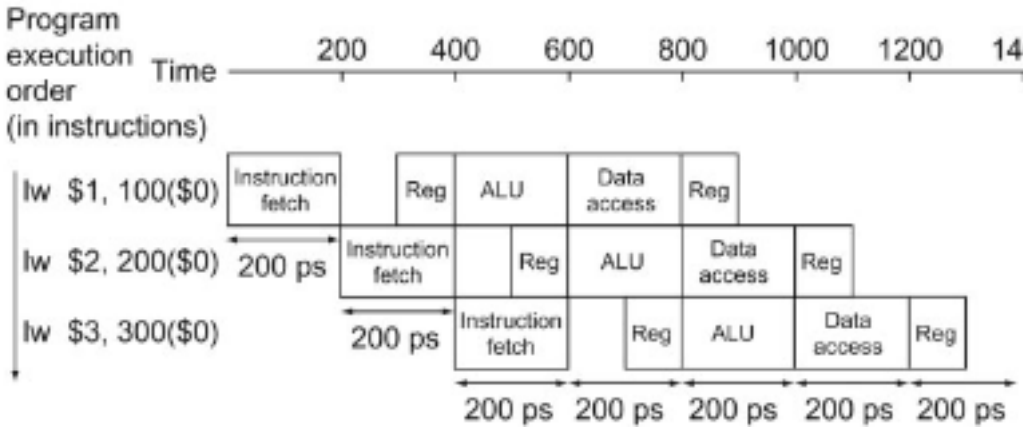
3.2. Pipeline

Dividir a instrução em etapas, cada etapa vai ser executada em 1 ciclo de clock, com sobreposição de etapas diferentes, de instruções diferentes, no mesmo ciclo de clock. Ideia de linha de montagem.

Etapas

- IF: Instruction Fetch - Mem. Instr.
- ID: Instruction Decode - UC/Banco Reg.
- EX: Execution - ULA
- Mem: Memory Access - Mem. Dados
- WB: Write Back - Banco de Registradores





Comparação entre Monociclo, Multiciclo e Pipeline

- Monociclo: Cada instrução é executada completa em um único ciclo de clock. O ciclo precisa ser longo o suficiente para aguentar a instrução mais lenta.
- Multiciclo: Divide a execução de uma instrução em várias etapas menores. O ciclo é curto e baseado na etapa mais longa. instruções diferentes usam um número diferente de ciclos. Uma nova instrução só começa quando a anterior termina.
- Pipeline: Divida a instrução em estágios, mas permite sobreposição.

Para 100 instruções:

Obs: IPC = instruction per clock, CPI = clock per instruction

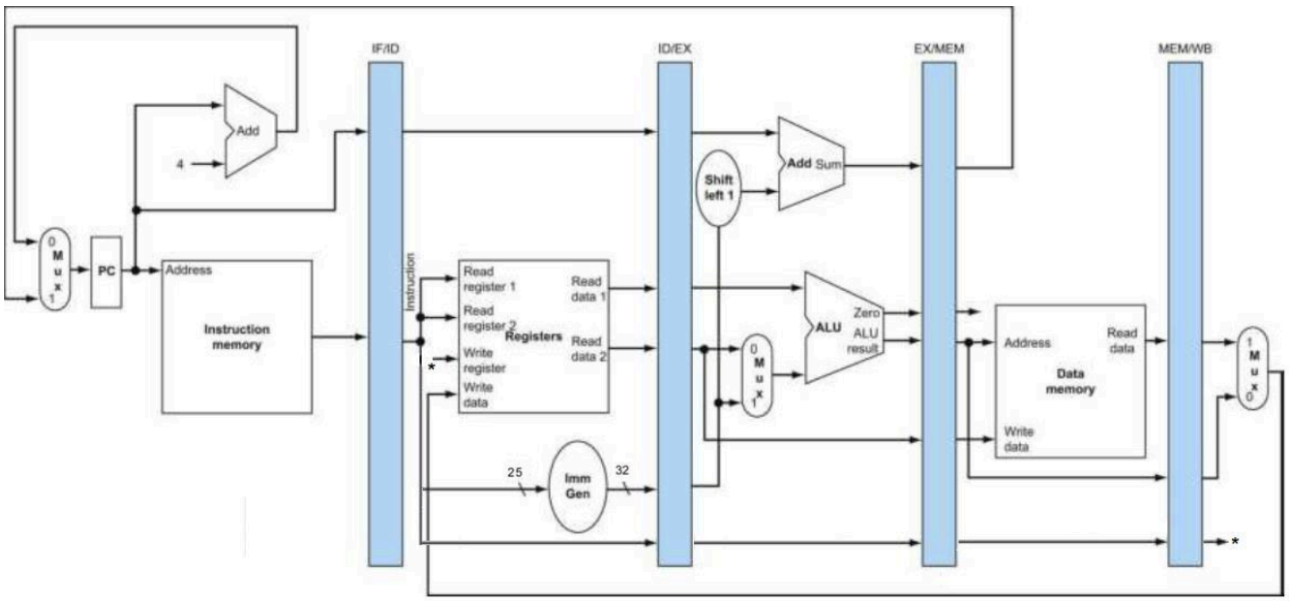
- Monociclo:
  - Tempo de clock = 45 ns
  - Para 100 instruções: $45 \cdot 100 = 4500$ ns
- Multiciclo:
  - Tempo de clock = 10 ns
  - CPI médio = 4.2 clocks/inst.
  - Para 100 instruções: $4.2 \cdot 10 \cdot 100 = 4200$ ns
- Pipeline:
  - Tempo de ciclo de clock = 10 ns
  - N° de ciclos de clock = (1 inst: 5, 2 inst: 6, 3 inst: 7). Isso pois precisa primeiro “encher” o pipeline.
  - Para 100 instruções: 104 ciclos de clock - $104 \cdot 10 = 1040$ ns
  - Problema: paradas (gargalos - Instruções esperando liberar um componente, disputa por recursos)

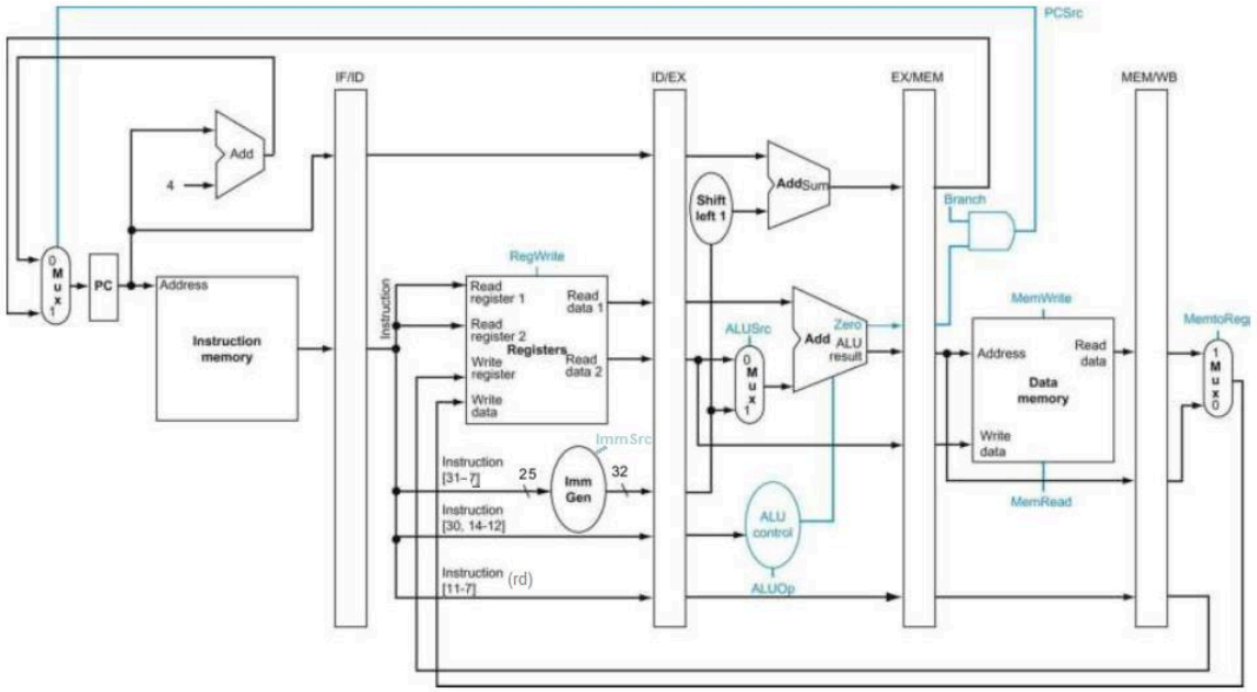
o registrador.

Considerações

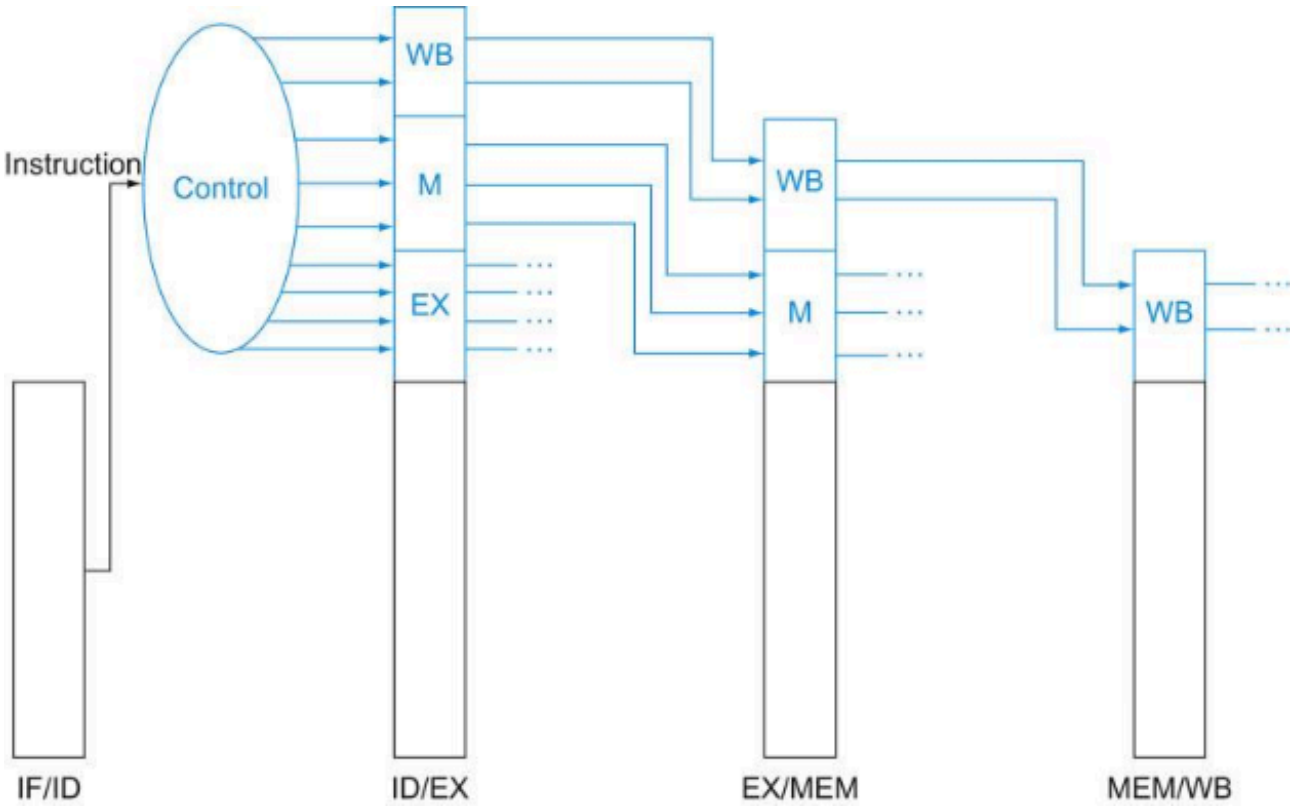
- O pipeline não diminui o tempo de execução de cada tarefa, mas reaproveita módulos não utilizados (aumenta throughput)
- A taxa do pipeline é limitada pelo estágio mais lento
- O pipeline trava se houver dependências

Registradores intermediários do pipeline





Sinais de controle:


Instruction
Control
WB
M
EX
WB
M
WB
IF/ID
ID/EX
EX/MEM
MEM/WB